

بسمه تعالی

درس سیستم های نهفته - نیمسال تحصیلی ۰۴-۰۵
دانشگاه صنعتی شریف - دانشکده مهندسی برق



گزارش بخش پنجم

امیرحسین مطیعی - شماره دانشجویی ۴۰۱۱۰۲۵۵۳

گزارش بخش پنجم توسعه API برای خواندن لاگ سنسورها

چکیده

در بخش پنجم تمرین، یک API مبتنی بر پروتکل HTTP برای بازیابی لاگ‌های ثبت‌شده سنسورها طراحی و پیاده‌سازی شد. هدف اصلی این API آن است که اپراتور بتواند با ارسال نام سنسور، شناسه سنسور و یک تاریخ مشخص، تمام مقادیر ثبت‌شده آن سنسور در تاریخ موردنظر را دریافت کند. مطابق محدودیت‌های تمرین، برنامه با زبان ++C نوشته شد و برای پیاده‌سازی سرور HTTP از کتابخانه سبک و رویدادمحور **Mongoose** استفاده گردید. اطلاعات سنسورها و مقادیر ثبت‌شده نیز مستقیماً از پایگاه داده **SQLite** خوانده می‌شوند.

API روی ماشین مجازی **Master** اجرا شد و به پایگاه داده واقعی این نود در مسیر `home/amir/master/master.db` متصل گردید. برای جلوگیری از وابستگی برنامه به محیط آزمایش، آدرس `Listen`، شماره پورت و مسیر پایگاه داده داخل کد ثابت نشدند و از فایل پیکربندی دریافت شدند. درخواست‌ها از طریق متد `GET` و مسیر `/api/sensor-logs` ارسال می‌شوند و پاسخ‌ها در قالب **JSON** خوانا برگردانده می‌شوند.

برای ارزیابی کامل پیاده‌سازی، تست‌های ساختار پروژه، ساختار پایگاه داده، کامپایل با `Makefile`، اجرای برنامه با اسکریپت `Bash`، سلامت سرویس، دسترسی از `Host` به ماشین مجازی **Master**، بازیابی داده‌های سنسورهای دما، رطوبت و حرکت، اعتبارسنجی ورودی‌ها، نبود داده، تطبیق نام و شناسه سنسور، مسیر نامعتبر، متد نامعتبر، اجرای اسکریپت خودکار و توقف صحیح سرویس انجام شد. نتایج نشان دادند که API در تمامی حالت‌های اصلی و خطاهای مورد انتظار رفتار صحیح و قابل پیش‌بینی دارد.

مقدمه

در بخش‌های قبلی پروژه، زیرساخت لازم برای نگهداری و خواندن داده‌های سنسورهای هتل ایجاد شد. هر سنسور دارای اطلاعاتی مانند شناسه، نوع، نام، محل نصب و واحد اندازه‌گیری است و مقادیر اندازه‌گیری شده آن در زمان‌های مختلف در پایگاه داده ذخیره می‌شوند. در بخش پنجم، نیاز جدیدی مطرح شد که بر اساس آن اپراتور باید بتواند به‌جای دریافت صرفاً آخرین مقدار سنسور، تمام مقادیر ثبت‌شده آن سنسور را در یک روز مشخص مشاهده کند.

برای پاسخ‌گویی به این نیاز، یک API مستقل روی ماشین مجازی **Master** ایجاد شد. این API نقش واسط میان اپراتور و پایگاه داده **SQLite** را بر عهده دارد. اپراتور لازم نیست ساختار داخلی پایگاه داده، نام جدول‌ها یا نحوه اجرای `Query` را بداند و تنها سه پارامتر مشخص را ارسال می‌کند. سرور پس از اعتبارسنجی درخواست، داده‌های مرتبط را از پایگاه داده استخراج کرده و آن‌ها را در قالب یک پاسخ استاندارد **JSON** برمی‌گرداند.

استفاده از API باعث جداسازی لایه ارتباط شبکه از لایه ذخیره سازی می شود. در این ساختار، مشتری یا اپراتور فقط با رابط HTTP سروکار دارد و جزئیات خواندن از SQLite در داخل برنامه پنهان می ماند. این طراحی در آینده امکان اتصال داشبوردهای مانیتورینگ، برنامه های موبایل، نرم افزارهای مدیریتی یا سایر سرویس ها به داده های سنسورها را فراهم می کند.

تحلیل نیازمندی های بخش پنجم

نیاز اصلی بخش پنجم، پیاده سازی یک API با استفاده از Mongoose و زبان C یا C++ بود API. باید سه ورودی شامل نام سنسور، شناسه سنسور و تاریخ موردنظر را دریافت کند. سپس باید تمام مقادیر ثبت شده سنسور در آن تاریخ را از پایگاه داده بخواند و در یک ساختار مشخص و قابل فهم بازگرداند. در صورت نبود داده نیز باید یک پیام مناسب به کاربر ارائه شود.

علاوه بر عملکرد اصلی، الزامات عمومی تمرین نیز در طراحی لحاظ شدند. برای کامپایل برنامه از Makefile استفاده شد. یک اسکریپت Bash برای کامپایل و اجرای API تهیه گردید. یک اسکریپت جداگانه برای ایجاد پایگاه داده آزمایشی و یک اسکریپت برای تست خودکار API نیز در پروژه قرار گرفت. نحوه نصب وابستگی ها، کامپایل، اجرا، ارسال درخواست و انجام تست ها در فایل README.md توضیح داده شد.

یکی از مهم ترین محدودیت های تمرین این بود که مقادیر اجرایی نظیر IP، پورت و مسیر پایگاه داده نباید در کد ثابت باشند. به همین دلیل، آدرس Listen و مسیر پایگاه داده از فایل config/api.env دریافت می شوند. همچنین داده های پاسخ به صورت ثابت داخل کد قرار نگرفته اند و در هر درخواست، اطلاعات واقعی از پایگاه داده SQLite خوانده می شوند. این موضوع باعث می شود برنامه در زمان تصحیح نیز با فایل های داده جدید قابل استفاده باشد.

معماری کلی سیستم

در این بخش، API روی ماشین مجازی Master اجرا می شود Host. نقش اپراتور یا Client را دارد و از طریق شبکه محلی یک درخواست HTTP به IP ماشین Master و پورت ۸۰۰۵ ارسال می کند. برنامه Mongoose درخواست را دریافت کرده، مسیر و پارامترها را بررسی می کند و سپس لایه دسترسی به داده را فراخوانی می نماید. لایه دسترسی به داده، پایگاه داده SQLite را به صورت فقط خواندنی باز کرده و با استفاده از Query پارامتری، مقادیر منطبق را استخراج می کند. نتیجه در قالب JSON به برنامه اصلی بازگردانده شده و از طریق اتصال HTTP برای Host ارسال می شود.

دیگرام مسیر درخواست و پاسخ به صورت زیر است:

```
Host operator
|
| HTTP GET over the local network
v
Master VM: Mongoose API on port 8005
|
| Prepared read-only SQLite query
v
/home/amir/master/master.db
|
| JSON response
v
Host operator
```

در این معماری، ماشین‌های Slave1 و Slave2 مستقیماً در اجرای بخش پنجم دخالت ندارند، زیرا API آزمایش‌شده به پایگاه داده محلی Master متصل است. با این حال، لایه دسترسی به داده به گونه‌ای طراحی شده است که در صورت نیاز بتوان بیش از یک مسیر پایگاه داده را از طریق تنظیمات به آن معرفی کرد. بنابراین توسعه آینده برای خواندن از چند فایل SQLite بدون تغییر اساسی معماری امکان‌پذیر است.

ساختار پروژه

پوشه بخش پنجم به صورت مستقل از سایر بخش‌های تمرین سازمان‌دهی شد. ساختار نهایی پروژه به شکل زیر است:

```

===== SECTION 05 PROJECT STRUCTURE =====
.
├── api
│   ├── main.cpp
│   └── mongoose
│       ├── .gitkeep
│       ├── mongoose.c
│       └── mongoose.h
│   ├── sensor_log_store.cpp
│   └── sensor_log_store.hpp
├── config
│   ├── api.env
│   └── api.env.example
├── Makefile
├── README.md
├── scripts
│   ├── build_and_run.sh
│   ├── create_test_db.sh
│   ├── fetch_mongoose.sh
│   └── test_api.sh
└── tests
    └── test_store.cpp

6 directories, 15 files

```

فایل `main.cpp` مسئول ایجاد مدیر رویداد `Mongoose`، راه‌اندازی `Listener`، شناسایی مسیر درخواست، استخراج پارامترهای `Query`، اعتبارسنجی ورودی‌ها و ارسال پاسخ HTTP است. منطق مربوط به `SQLite` از سرور HTTP جدا شده و در فایل `sensor_log_store.cpp` قرار دارد. این جداسازی باعث می‌شود لایه دسترسی به داده بدون اجرای سرور شبکه قابل آزمایش باشد.

فایل‌های `mongoose.c` و `mongoose.h` نسخه ۷.۲۲ کتابخانه `Mongoose` هستند و به صورت مستقیم همراه پروژه کامپایل می‌شوند. بنابراین در زمان کامپایل عادی، نیازی به نصب یک بسته سیستمی جداگانه برای `Mongoose` وجود ندارد.

فایل `Makefile` هدف‌های لازم برای پاک‌سازی فایل‌های تولیدشده، ساخت API و اجرای تست مستقل پایگاه داده را در اختیار قرار می‌دهد. اسکریپت `build_and_run.sh` تنظیمات را از فایل محیطی می‌خواند، پروژه را کامپایل می‌کند و API را اجرا می‌نماید. اسکریپت `create_test_db.sh` یک پایگاه داده موقت برای تست مستقل ایجاد می‌کند و اسکریپت `test_api.sh` سلامت سرور و یک درخواست نمونه را بررسی می‌کند. توضیح کامل اجزای پروژه و روش اجرا نیز در `README` بخش پنجم ثبت شده است.

مدیریت تنظیمات اجرایی

تنظیمات اصلی API در فایل `config/api.env` نگهداری می‌شوند. محتوای تنظیمات استفاده‌شده در محیط آزمایش به صورت زیر بود:

```
API_LISTEN_URL=http://0.0.0.0:8005
```

```
DATABASE_PATHS=/home/amir/master/master.db
```

```
MONGOOSE_VERSION=7.22
```

متغیر `API_LISTEN_URL` مشخص می کند سرور روی کدام `Port` و `Interface` اجرا شود. استفاده از `0.0.0.0` باعث می شود سرور علاوه بر درخواست های محلی، درخواست های ورودی از `Host` را نیز بپذیرد. اگر این مقدار روی `127.0.0.1` تنظیم می شد، `API` فقط از داخل همان ماشین مجازی قابل دسترسی بود.

متغیر `DATABASE_PATHS` مسیر پایگاه داده ای را مشخص می کند که `API` باید از آن بخواند. در آزمایش نهایی، پایگاه داده واقعی `Master` در مسیر `/home/amir/master/master.db` قرار داشت. مسیر دیگری با نام `home/amir/master.db` نیز روی ماشین وجود داشت، اما فایل آن صفر بایت بود و به همین دلیل در تنظیمات استفاده نشد.

متغیر `MONGOOSE_VERSION` نسخه مورد استفاده توسط اسکریپت دریافت کتابخانه را تعیین می کند. نسخه موجود و آزمایش شده در پروژه `۷.۲۲` بود. انتقال این مقادیر به فایل پیکربندی باعث می شود در صورت تغییر `IP`، پورت یا محل پایگاه داده، نیازی به ویرایش و کامپایل مجدد کد `C++` نباشد.

ساختار پایگاه داده

پایگاه داده `Master` از نوع `SQLite` است و سه جدول `node_info`، `sensors` و `sensor_readings` را شامل می شود. دو جدول اصلی مورد استفاده در بخش پنجم، `sensors` و `sensor_readings` هستند.

جدول `sensors` اطلاعات ثابت هر سنسور را نگهداری می کند. شناسه سنسور در ستون `sensor_id` قرار دارد و به عنوان کلید اصلی استفاده می شود. نوع سنسور در `sensor_type` و نام کامل آن در `sensor_name` ذخیره شده است. اطلاعاتی مانند محل نصب، واحد اندازه گیری، نام نود و وضعیت فعال بودن نیز در همین جدول قرار دارند.

جدول `sensor_readings` مقادیر اندازه گیری شده سنسورها را نگهداری می کند. هر رکورد شامل شناسه سنسور، مقدار ثبت شده، زمان ثبت و زمان ایجاد رکورد است. ارتباط میان این جدول و جدول `sensors` از طریق ستون `sensor_id` برقرار می شود.

نمایش جدول ها و ساختار ستون های پایگاه داده SQLite

```
===== DATABASE FILE =====
-rw-r--r-- 1 amir amir 32K Jul 21 12:50 /home/amir/master/master.db

===== DATABASE TABLES =====
node_info      sensor_readings  sensors

===== SENSORS TABLE STRUCTURE =====
cid  name      type      notnull  dflt_value  pk
---  -
0    sensor_id  TEXT      0        0            1
1    sensor_type TEXT      1        0            0
2    sensor_name TEXT      1        0            0
3    location   TEXT      0        0            0
4    unit       TEXT      0        0            0
5    node_name  TEXT      1        0            0
6    is_active  INTEGER   1        1            0

===== SENSOR_READINGS TABLE STRUCTURE =====
cid  name      type      notnull  dflt_value  pk
---  -
0    id        INTEGER   0        0            1
1    sensor_id TEXT      1        0            0
2    value     TEXT      1        0            0
3    recorded_at TEXT      1        0            0
4    created_at TEXT      0        CURRENT_TIMESTAMP 0

amir@embedded-master: $
```

برای پاسخ گویی به یک درخواست، `API` جدول `sensor_readings` را با جدول `sensors` بر اساس شناسه سنسور `Join` می کند. شرط جست و جو شامل شناسه سنسور، نام یا نوع سنسور و بخش تاریخ ستون `recorded_at` است. نتایج براساس زمان ثبت به صورت صعودی مرتب می شوند تا ترتیب وقایع روز از قدیمی ترین مقدار تا جدیدترین مقدار در پاسخ حفظ شود.

نام ارسالی می تواند برابر نوع سنسور، مانند `temperature`، یا نام کامل ذخیره شده، مانند `Floor1_Room101_Temp`، باشد. مقایسه نام ها بدون حساسیت به حروف کوچک و بزرگ انجام می شود. با این حال، شناسه و نام باید به یک سنسور واحد تعلق داشته باشند و صرفاً وجود یکی از آن ها برای بازگرداندن داده کافی نیست.

پیش از اجرای API، داده های واقعی پایگاه داده بررسی شدند. پایگاه داده Master شامل چهار سنسور با شناسه های ۱۰۱ تا ۱۰۴ بود. برای هر سنسور دو مقدار در تاریخ اول ژوئن ۲۰۲۶ ثبت شده بود. سنسورهای موجود شامل دو سنسور دما، یک سنسور رطوبت و یک سنسور حرکت بودند.

نمایش داده های واقعی سنسورها در پایگاه داده Master

```
===== REAL SENSOR LOG DATA =====
sensor_id  sensor_type  sensor_name  value  reading_date  reading_time
-----
101         temperature  Floor1_Room101_Temp  24.2   2026-06-01   10:00:00
101         temperature  Floor1_Room101_Temp  24.8   2026-06-01   10:15:00
102         humidity     Floor1_Room101_Humidity  44     2026-06-01   10:00:00
102         humidity     Floor1_Room101_Humidity  45     2026-06-01   10:15:00
103         motion       Floor1_Corridor_Motion  0       2026-06-01   10:10:00
103         motion       Floor1_Corridor_Motion  1       2026-06-01   10:16:00
104         temperature  Floor1_Lobby_Temp      23.5   2026-06-01   10:05:00
104         temperature  Floor1_Lobby_Temp      23.9   2026-06-01   10:20:00
amir@embedded-master:~$
```

این تصویر نشان می دهد مقادیری که بعداً در خروجی API دیده می شوند، از رکوردهای واقعی SQLite استخراج شده اند و پاسخ های ثابت یا از پیش آماده شده در برنامه وجود ندارند.

طراحی API

API اصلی در مسیر زیر تعریف شده است:

GET /api/sensor-logs

این Endpoint سه پارامتر Query دریافت می کند:

sensor_name

sensor_id

date

Required query parameters:

Parameter	Description	Example
sensor_name	Sensor type or stored sensor name	temperature
sensor_id	Sensor identifier	101
date	Requested date in YYYY-MM-DD format	2026-06-01

یک درخواست نمونه به شکل زیر است:

/api/sensor-logs?sensor_name=temperature&sensor_id=101&date=2026-06-01

پارامتر `sensor_name` می تواند نوع سنسور یا نام کامل ذخیره شده آن باشد. پارامتر `sensor_id` شناسه یکتای سنسور را مشخص می کند و پارامتر `date` باید یک تاریخ معتبر با قالب YYYY-MM-DD باشد.

پاسخ موفق با کد وضعیت 200 OK و قالب JSON برگردانده می شود:

```
{
  "sensor_name": "temperature",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:15:00",
      "value": "24.8"
    },
    {
      "time": "10:30:00",
      "value": "25.1"
    }
  ]
}
```

در سطح بالای پاسخ، اطلاعات درخواست تکرار می شوند تا Client بتواند تشخیص دهد پاسخ متعلق به کدام سنسور و تاریخ است. آرایه values شامل تمام مقادیر پیداشده است و هر عضو آن زمان و مقدار یک Reading را در اختیار قرار می دهد.

علاوه بر Endpoint اصلی، مسیر زیر برای بررسی سلامت سرویس تعریف شد:

GET /health

این مسیر بدون مراجعه به داده های سنسور، وضعیت فعال بودن برنامه و قابلیت دریافت درخواست HTTP را نشان می دهد. پاسخ صحیح آن به صورت زیر است:

```
{
  "status": "ok"
}
```

وجود Health Check برای تست سریع، مانیتورینگ و تشخیص تفاوت میان خرابی سرور و نبود داده مفید است.

مسیر پردازش درخواست

هنگامی که یک درخواست HTTP به سرور می رسد، Mongoose رویداد دریافت پیام HTTP را ایجاد می کند. برنامه ابتدا متد درخواست را کنترل می کند و فقط متد GET را برای Endpoint های تعریف شده می پذیرد. سپس URI درخواست با مسیرهای /health و /api/sensor-logs مقایسه می شود.

در درخواست خواندن لاگ، سه پارامتر Query استخراج می شوند. اگر هر کدام وجود نداشته باشند، درخواست پیش از دسترسی به پایگاه داده رد می شود. در مرحله بعد، تاریخ بررسی می شود تا هم قالب آن صحیح باشد و هم یک تاریخ واقعی تقویمی را نشان دهد. برای نمونه، تاریخ 2026-02-30 از نظر ظاهری شبیه قالب صحیح است، اما چون سی ام فوریه وجود ندارد، معتبر شناخته نمی شود.

پس از تأیید ورودی ها، لایه پایگاه داده فایل SQLite را در حالت Read-only باز می کند. Query به صورت Prepared Statement اجرا می شود و مقادیر ورودی به پارامترهای آن Bind می شوند. این روش نسبت به ساخت مستقیم رشته SQL امن تر است و احتمال تزریق SQL را کاهش می دهد.

اگر رکورد منطبق پیدا شود، داده ها به ترتیب زمانی در JSON قرار می گیرند و با کد 200 ارسال می شوند. اگر Query با موفقیت اجرا شود ولی رکوردی با ترکیب نام، شناسه و تاریخ وجود نداشته باشد، آرایه values خالی همراه با پیام مناسب و کد 404 برگردانده می شود. اگر پایگاه داده قابل باز شدن یا Query قابل اجرا نباشد، پاسخ خطای داخلی سرور تولید می شود.

کامپایل و تست مستقل لایه پایگاه داده

برای کامپایل API از Makefile استفاده شد. دستور `make clean` فایل های اجرایی و پایگاه داده موقت تست را حذف می کند و دستور `make all` برنامه اصلی را با استاندارد C++17 کامپایل می نماید. فایل های `main.cpp`، `sensor_log_store.cpp` و `mongoose.c` در فرایند ساخت شرکت می کنند و برنامه با کتابخانه SQLite و Thread پیوند داده می شود.

فایل اجرایی نهایی در مسیر زیر ایجاد شد:

`api/sensor_log_api`

پیش از اجرای سرور HTTP، لایه پایگاه داده به صورت مستقل با دستور `make test-core` آزمایش شد. در این تست، اسکریپت `create_test_db.sh` یک پایگاه داده موقت به نام `tests/test_sensor.db` ایجاد کرد و برنامه `tests/test_store` بدون استفاده از Mongoose، داده سنسور ۱۰۱ را از آن خواند.

در پایگاه داده آزمایشی، مقادیر 24.8 در ساعت 10:15:00 و 25.1 در ساعت 10:30:00 قرار داشتند. بازیابی صحیح این دو مقدار نشان داد که منطق Join، فیلتر تاریخ، مرتب سازی زمان و تولید JSON مستقل از سرور شبکه درست کار می کند.

کامپایل تمیز پروژه با Makefile و اجرای موفق تست مستقل SQLite

```
===== CLEAN BUILD =====
rm -f api/sensor_log_api tests/test_store tests/test_sensor.db

===== BUILD API WITH MAKEFILE =====
g++ -Iapi -std=c++17 -O2 -Wall -Wextra -Wpedantic api/main.cpp api/sensor_log_store.cpp api/mongoose/mongoose.c -lsqlite3 -pthread -o api/sensor_log_api

===== CORE SQLITE TEST =====
g++ -Iapi -std=c++17 -O2 -Wall -Wextra -Wpedantic tests/test_store.cpp api/sensor_log_store.cpp -lsqlite3 -o tests/test_store
./scripts/create_test_db.sh tests/test_sensor.db
Created test database: tests/test_sensor.db
./tests/test_store tests/test_sensor.db
{
  "sensor_name": "temperature",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:15:00",
      "value": "24.8"
    },
    {
      "time": "10:30:00",
      "value": "25.1"
    }
  ]
}

===== CREATED EXECUTABLE =====
-rwxrwxr-x 1 amir amir 242K Jul 21 21:43 api/sensor_log_api
amir@embedded-master:~/BSE$
```

استفاده از یک پایگاه داده موقت باعث شد لایه خواندن داده جدا از پایگاه داده واقعی Master آزمایش شود. این موضوع نشان می دهد کد وابستگی ثابتی به داده های محیط اصلی ندارد و می تواند با فایل های SQLite دیگری که ساختار مشابه دارند نیز کار کند.

اجرای API با اسکریپت Bash

برای اجرای نهایی برنامه از اسکریپت `scripts/build_and_run.sh` استفاده شد. این اسکریپت ابتدا فایل پیکربندی را بارگذاری می کند و وجود متغیرهای ضروری را بررسی می نماید. سپس فایل های Mongoose را کنترل کرده، پروژه را با Makefile کامپایل می کند و برنامه را با آرگومان های Listen و مسیر پایگاه داده اجرا می نماید.

API در حالت Foreground اجرا می شود تا پیام های شروع و خطاهای احتمالی مستقیماً در ترمینال دیده شوند. در آزمایش انجام شده، سرویس روی نشانی زیر فعال شد:

http://0.0.0.0:8005

و منبع SQLite آن برابر مسیر زیر بود:

/home/amir/master/master.db

اجرای اسکریپت Bash برای کامپایل و راه اندازی API

```
===== BUILD AND RUN SCRIPT =====
make: Entering directory '/home/amir/05'
rm -f api/sensor_log_api tests/test_store tests/test_sensor.db
g++ -Iapi -std=c++17 -O2 -Wall -Wextra -Wpedantic api/main.cpp api/sensor_log_store.cpp api/mongoose/mongoose.c -lsqlite3 -pthread -o api/sensor_log_api
make: Leaving directory '/home/amir/05'
a70b9 3 mongoose.c:7806: MG_IO_SIZE: 16384, TLS: none
a70b9 3 mongoose.c:7719: 1 4 http://0.0.0.0:8005
Sensor log API is listening on http://0.0.0.0:8005
SQLite source: /home/amir/master/master.db
```

پس از اجرا، وجود Process برنامه و باز بودن پورت ۸۰۰۵ با دستورات سیستم عامل بررسی شد. نتیجه نشان داد برنامه `sensor_log_api` در حال اجراست و روی تمام Interface های IPv4 به درخواست ها گوش می دهد. سپس Endpoint سلامت با `curl` فراخوانی شد و پاسخ 200 OK همراه با وضعیت ok دریافت گردید.

بررسی Process، پورت Listen و پاسخ Health Check

```
===== RUNNING API PROCESS =====
1299 /home/amir/05/api/sensor_log_api --listen http://0.0.0.0:8005 --db /home/amir/master/master.db

===== LISTENING PORT =====
LISTEN 0      128      0.0.0.0:8005  0.0.0.0:*    users: (("sensor_log_api",pid=1299,fd=4))

===== HEALTH CHECK =====
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 21

{
  "status": "ok"
}
amir@embedded-master: ~$
```

این تست نشان داد سرور Mongoose به درستی راه اندازی شده و قبل از شروع آزمایش های داده، قابلیت دریافت و پاسخ گویی به درخواست های HTTP را دارد.

تست ارتباط شبکه از Host به Master

یکی از اهداف مهم آزمایش، اطمینان از این بود که API فقط از داخل ماشین Master قابل استفاده نباشد. به همین دلیل، درخواست اصلی از Host به IP ماشین مجازی Master ارسال شد. IP ماشین 192.168.122.22 بود و درخواست به پورت ۸۰۰۵ ارسال گردید.

در این تست، Host درخواست مربوط به سنسور دمای ۱۰۱ در تاریخ 2026-06-01 را ارسال کرد. پاسخ شامل دو مقدار 24.2 و 24.8 بود و کد وضعیت 200 OK دریافت شد.

ارسال درخواست از Host به API ماشین Master و دریافت پاسخ JSON

```

===== HOST TO MASTER NETWORK TEST =====
Master API: http://192.168.122.22:8005

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 221

{
  "sensor_name": "temperature",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:00:00",
      "value": "24.2"
    },
    {
      "time": "10:15:00",
      "value": "24.8"
    }
  ]
}
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04$

```

موفقیت این تست، مسیر کامل زیر را تأیید کرد:

Host → Local Network → Master VM → Mongoose API → SQLite → JSON → Host

در نتیجه، تنظیم Listen روی 0.0.0.0 و ارتباط شبکه محلی به درستی عمل کرده است. IP نشان داده شده متعلق به محیط آزمایش است و داخل کد ثابت نشده است؛ در صورت تغییر IP ماشین مجازی، Client باید IP جدید را در درخواست استفاده کند.

تست بازیابی داده انواع مختلف سنسور

برای اطمینان از اینکه API به یک نوع خاص از سنسور محدود نیست، درخواست هایی برای سنسورهای دما، رطوبت و حرکت ارسال شدند. در تست سنسور دما با شناسه ۱۰۱، دو مقدار 24.2 و 24.8 به ترتیب در ساعت های 10:00:00 و 10:15:00 بازگردانده شدند.

پاسخ API برای سنسور دما با شناسه ۱۰۱

```

===== TEMPERATURE SENSOR 101 =====
{
  "sensor_name": "temperature",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:00:00",
      "value": "24.2"
    },
    {
      "time": "10:15:00",
      "value": "24.8"
    }
  ]
}

```

در تست سنسور رطوبت با شناسه ۱۰۲، مقادیر 44 و 45 در ساعت های 10:00:00 و 10:15:00 دریافت شدند.

پاسخ API برای سنسور رطوبت با شناسه ۱۰۲

```
===== HUMIDITY SENSOR 102 =====
{
  "sensor_name": "humidity",
  "sensor_id": "102",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:00:00",
      "value": "44"
    },
    {
      "time": "10:15:00",
      "value": "45"
    }
  ]
}
```

در تست سنسور حرکت با شناسه ۱۰۳، مقادیر صفر و یک در ساعت‌های 10:10:00 و 10:16:00 بازگردانده شدند. صفر نشان‌دهنده نبود حرکت و یک نشان‌دهنده تشخیص حرکت است.

پاسخ API برای سنسور حرکت با شناسه ۱۰۳

```
===== MOTION SENSOR 103 =====
{
  "sensor_name": "motion",
  "sensor_id": "103",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:10:00",
      "value": "0"
    },
    {
      "time": "10:16:00",
      "value": "1"
    }
  ]
}
amir@embedded-master:~$
```

نتایج این سه تست نشان دادند که مقدار سنسور در API به نوع داده عددی خاصی محدود نشده و مقادیر متنی ذخیره‌شده در SQLite بدون تغییر نامناسب در پاسخ JSON قرار می‌گیرند.

مدیریت حالت نبود داده

یکی از الزامات صریح بخش پنجم، ارائه پیام مناسب در صورت نبود داده برای تاریخ درخواست‌شده بود. برای آزمایش این حالت، درخواست سنسور دمای ۱۰۱ برای تاریخ 2026-06-02 ارسال شد، در حالی که داده‌های موجود متعلق به روز قبل بودند.

API درخواست را از نظر ساختار معتبر تشخیص داد، اما هیچ Reading مطابقی در پایگاه داده پیدا نکرد. در نتیجه، پاسخ 404 Not Found برگردانده شد و آرایه values خالی بود. پیام No data found for the requested sensor and date نیز دلیل خالی بودن نتیجه را مشخص کرد.

پاسخ ۴۰۴ و پیام مناسب در صورت نبود داده برای تاریخ درخواست شده

```

===== NO DATA FOR REQUESTED DATE =====
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 161

{
  "sensor_name": "temperature",
  "sensor_id": "101",
  "date": "2026-06-02",
  "values": [],
  "message": "No data found for the requested sensor and date"
}
amir@embedded-master:~$

```

استفاده از 404 در این حالت به Client اجازه می دهد نبود داده را از خطای ورودی یا خرابی داخلی سرویس تفکیک کند.

اعتبارسنجی تطابق نام و شناسه سنسور

برای جلوگیری از بازگرداندن داده اشتباه، API تنها بر اساس شناسه جست و جو نمی کند، بلکه نام یا نوع سنسور نیز باید با همان رکورد مطابقت داشته باشد. برای آزمایش این ویژگی، شناسه ۱۰۱ که متعلق به سنسور دماست همراه با نام humidity ارسال شد.

اگر API تنها شناسه را در نظر می گرفت، ممکن بود مقادیر دما را برای درخواستی با نام رطوبت بازگرداند. اما در طراحی انجام شده، ترکیب نام و شناسه بررسی شد و چون این دو به یک سنسور تعلق نداشتند، پاسخ بدون داده و با وضعیت 404 بازگردانده شد.

رد درخواست دارای عدم تطابق میان نام و شناسه سنسور

```

===== SENSOR NAME AND ID MISMATCH =====
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 158

{
  "sensor_name": "humidity",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [],
  "message": "No data found for the requested sensor and date"
}
amir@embedded-master:~$

```

این رفتار از نظر صحت داده اهمیت زیادی دارد، زیرا مانع از آن می شود که Client در اثر ارسال پارامترهای متناقض، داده متعلق به سنسور دیگری را دریافت کند.

اعتبارسنجی تاریخ

تاریخ ورودی باید هم از قالب YYYY-MM-DD پیروی کند و هم از نظر تقویمی معتبر باشد. برای آزمایش این ویژگی، تاریخ 2026-02-30 ارسال شد. با وجود اینکه این مقدار از نظر تعداد کاراکترها و محل خط تیره ها شبیه قالب صحیح است، روز سی ام برای ماه فوریه وجود ندارد. API این مقدار را پیش از اجرای Query رد کرد و پاسخ 400 Bad Request را همراه با پیام زیر برگرداند:

```

{
  "error": "date must be a valid YYYY-MM-DD value"
}

```

پاسخ ۴۰۰ برای تاریخ تقویمی نامعتبر یا قالب تاریخ نادرست

```

===== INVALID CALENDAR DATE =====
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 55

{
  "error": "date must be a valid YYYY-MM-DD value"
}

===== INVALID DATE FORMAT =====
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 55

{
  "error": "date must be a valid YYYY-MM-DD value"
}
amir@embedded-master:~$

```

این اعتبارسنجی مانع از ارسال Query های بی معنا به پایگاه داده می شود و خطای کاربر را به شکل واضح توضیح می دهد.

مدیریت پارامترهای ناقص

هر سه پارامتر sensor_name، sensor_id و date برای اجرای درخواست ضروری هستند. در تست پارامتر ناقص، ابتدا تاریخ و سپس نام سنسور از درخواست حذف شد. در هر دو حالت، API پیش از ورود به لایه پایگاه داده، درخواست را رد کرد و وضعیت 400 Bad Request را برگرداند.

پیام خطا به صورت زیر بود:

```

{
  "error": "sensor_name, sensor_id and date query parameters are required"
}

```

پاسخ ۴۰۰ در صورت حذف یکی از پارامترهای اجباری API

```

===== MISSING DATE PARAMETER =====
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 79

{
  "error": "sensor_name, sensor_id and date query parameters are required"
}

===== MISSING SENSOR NAME PARAMETER =====
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 79

{
  "error": "sensor_name, sensor_id and date query parameters are required"
}
amir@embedded-master:~$

```

این پیام دقیقاً مشخص می کند چه ورودی هایی برای درخواست لازم هستند و از تولید پاسخ مبهم جلوگیری می نماید.

مدیریت مسیر و متد نامعتبر

برای تست Routing، درخواست به مسیر تعریف نشده /api/unknown ارسال شد. از آنجا که این مسیر در برنامه وجود نداشت، سرور پاسخ 404 Not Found همراه با پیام Route not found را برگرداند. در تست دیگری، متد POST برای Endpoint خواندن لاگ ارسال شد. طراحی API فقط متد GET را می پذیرد، زیرا عملیات موردنظر صرفاً خواندن داده است و تغییری در منابع ایجاد نمی کند. در نتیجه، سرور پاسخ 405 Method Not Allowed را برگرداند. وجود سرآیند Allow: GET نیز متد مجاز را به Client اعلام کرد.

مدیریت مسیر نامعتبر و متد HTTP غیرمجاز

```
===== UNKNOWN API ROUTE =====
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 33

{
  "error": "Route not found"
}

===== POST METHOD NOT ALLOWED =====
HTTP/1.1 405 Method Not Allowed
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Allow: GET
Content-Length: 48

{
  "error": "Only the GET method is allowed"
}
amir@embedded-master:~$
```

تفکیک میان 404 و 405 نشان می دهد برنامه میان نبود Endpoint و استفاده اشتباه از یک Endpoint موجود تفاوت قائل می شود.

جست و جو با نام کامل ذخیره شده سنسور

علاوه بر نوع عمومی سنسور، API می تواند نام کامل ذخیره شده در جدول sensors را نیز دریافت کند. برای آزمایش این ویژگی، نام Floor1_Room101_Temp همراه با شناسه ۱۰۱ و تاریخ موردنظر ارسال شد.

درخواست با موفقیت اجرا شد و همان دو مقدار سنسور دمای اتاق ۱۰۱ بازگردانده شدند.

بازیابی داده با استفاده از نام کامل ذخیره شده سنسور

```
===== QUERY USING STORED SENSOR NAME =====
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
Content-Length: 229

{
  "sensor_name": "Floor1_Room101_Temp",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:00:00",
      "value": "24.2"
    },
    {
      "time": "10:15:00",
      "value": "24.8"
    }
  ]
}
amir@embedded-master:~$
```

پشتیبانی همزمان از `sensor_type` و `sensor_name` باعث افزایش می دهد `Client`. می تواند یک دسته عمومی مانند `temperature` یا نام دقیق یک سنسور مانند `Floor1_Room101_Temp` را ارسال کند، به شرط آنکه شناسه نیز با همان سنسور مطابقت داشته باشد.

اجرای اسکریپت تست خودکار

اسکریپت `scripts/test_api.sh` برای تسهیل تست سریع بخش پنجم تهیه شد. این اسکریپت ابتدا `Endpoint` سلامت را فراخوانی می کند و سپس یک درخواست نمونه برای سنسور ۱۰۱ در تاریخ `2026-06-01` ارسال می نماید.

اسکریپت آدرس `API` را از فایل پیکربندی می خواند. اگر آدرس `Listen` برابر `0.0.0.0` باشد، برای درخواست محلی آن را به `127.0.0.1` تبدیل می کند؛ زیرا `0.0.0.0` آدرس `Listen` است و به عنوان آدرس مقصد `Client` استفاده نمی شود.

خروجی اسکریپت شامل پاسخ سلامت و مقادیر واقعی سنسور بود.

اجرای موفق اسکریپت خودکار تست API

```
===== AUTOMATED API TEST SCRIPT =====
{
  "status": "ok"
}

{
  "sensor_name": "temperature",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:00:00",
      "value": "24.2"
    },
    {
      "time": "10:15:00",
      "value": "24.8"
    }
  ]
}
```

amir@embedded-master:~/05\$

وجود این اسکریپت باعث می شود تصحیح کننده بتواند پس از راه اندازی `API`، با یک دستور ساده عملکرد اصلی آن را مشاهده کند.

ذخیره و انتقال نتایج تست

برای نگهداری مستندات آزمایش، خروجی تمام تست های اصلی در فایل `test-output/api_complete_test_results.txt` ذخیره شد. این فایل شامل اطلاعات `Process` و `Port`، پاسخ `Health Check`، درخواست های موفق سنسورهای دما، رطوبت و حرکت، درخواست با نام کامل سنسور، نبود داده، عدم تطابق نام و شناسه، تاریخ نامعتبر، پارامتر ناقص، مسیر ناشناخته و متد غیرمجاز است.

ایجاد فایل جامع نتایج و نمایش عنوان بخش های مختلف تست

```

===== SAVED TEST RESULT =====
-rw-rw-r-- 1 amir amir 3.6K Jul 21 21:53 /home/amir/05/test-output/api_complete_test_results.txt

===== TEST SECTIONS =====
===== PROCESS AND LISTEN ADDRESS =====
===== HEALTH CHECK =====
===== VALID TEMPERATURE REQUEST =====
===== VALID HUMIDITY REQUEST =====
===== VALID MOTION REQUEST =====
===== REQUEST USING STORED SENSOR NAME =====
===== NO DATA =====
===== SENSOR NAME AND ID MISMATCH =====
===== INVALID DATE =====
===== MISSING PARAMETER =====
===== UNKNOWN ROUTE =====
===== METHOD NOT ALLOWED =====
amir@embedded-master: ~/05$

```

پس از ایجاد فایل روی Master، نتیجه با استفاده از scp به Host منتقل شد تا همراه پروژه و گزارش نگهداری شود. انتقال خروجی به Host از آن جهت مفید است که کدهای اصلی پروژه روی Host مدیریت می شوند و فایل های نهایی تحویل نیز از همان محل آماده خواهند شد.

انتقال فایل نتایج از Master به Host و بررسی فایل دریافت شده

```

===== SECTION 05 HOST TEST FILES =====
api_complete_test_results.txt 3623 bytes

===== COMPLETE TEST RESULT LINE COUNT =====
174 test-output/api_complete_test_results.txt
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG: ~/embedded/embedded_hw04/05$

```

با این روش، نتایج تست تنها در محیط موقت ماشین مجازی باقی نماندند و یک نسخه از آن ها در پوشه بخش پنجم پروژه ذخیره شد.

توقف صحیح سرویس

پس از تکمیل همه آزمایش ها، API با ارسال سیگنال وقفه از طریق Ctrl+C متوقف شد. برنامه سیگنال توقف را دریافت کرده، حلقه Poll کتابخانه Mongoose را پایان می دهد و منابع مدیر رویداد را آزاد می کند.

پس از توقف، Process برنامه و وضعیت پورت ۸۰۰۵ بررسی شدند. نتیجه نشان داد Process دیگر در حال اجرا نیست و پورت نیز آزاد شده است.

توقف صحیح API و آزاد شدن Process و پورت ۸۰۰۵

```

===== API SHUTDOWN CHECK =====
OK: API process has stopped
OK: Port 8005 is free
amir@embedded-master: ~/05$

```

این تست نشان می دهد برنامه علاوه بر شروع صحیح، قابلیت خاتمه کنترل شده را نیز دارد و پس از خروج، Port یا Process بلااستفاده در سیستم باقی نمی گذارد.

تحلیل مدیریت خطاها

مدیریت خطا در API به چند سطح تقسیم شده است. خطاهای مربوط به Client، مانند حذف پارامترها یا وارد کردن تاریخ نامعتبر، با کد 400 پاسخ داده می شوند. نبود داده برای یک درخواست معتبر و همچنین مسیر ناشناخته با کد 404 مشخص می شود. استفاده از متد غیرمجاز با کد 405 پاسخ داده می شود. خطاهای داخلی، مانند نبود فایل پایگاه داده، نداشتن مجوز خواندن یا نامعتبر بودن ساختار SQLite، در دسته خطاهای سرور قرار می گیرند و باید با کد 500 گزارش شوند. در طراحی لایه داده، پایگاه داده به صورت Read-only می شود تا API نتواند به صورت ناخواسته داده های سنسورها را تغییر دهد. این تفکیک کدهای وضعیت باعث می شود Client بتواند علت شکست درخواست را به درستی تشخیص دهد. برای مثال، پاسخ 400 نشان می دهد ورودی باید اصلاح شود، پاسخ 404 نشان می دهد ورودی معتبر است اما منبع یا داده وجود ندارد، و پاسخ 500 نشان دهنده مشکلی در سمت سرویس یا پایگاه داده است.

ملاحظات امنیتی

API بخش پنجم در محیط آزمایشگاهی و شبکه محلی پیاده سازی شده است. ارتباط از HTTP ساده استفاده می کند و در نسخه فعلی رمزنگاری TLS وجود ندارد. بنابراین، در یک شبکه عمومی، داده ها و پارامترهای درخواست می توانند در معرض شنود قرار گیرند. برای استفاده عملی، قرار دادن سرویس پشت یک Reverse Proxy مانند Nginx و فعال سازی HTTPS مناسب خواهد بود.

API در نسخه فعلی احراز هویت ندارد و هر Client دارای دسترسی شبکه می تواند درخواست ارسال کند. در نسخه توسعه یافته می توان از API Key، Token یا سازوکارهای احراز هویت استفاده کرد. محدودسازی Firewall به IP های مجاز و Rate Limiting نیز می تواند از دسترسی غیرمجاز یا ارسال تعداد زیاد درخواست جلوگیری کند.

استفاده از Prepared Statement برای Query های SQLite خطر تزریق SQL را کاهش می دهد. همچنین باز کردن پایگاه داده در حالت فقط خواندنی، سطح دسترسی سرویس را محدود می کند. بررسی دقیق تاریخ و پارامترهای ضروری نیز از ورود داده های نامعتبر به لایه پایگاه داده جلوگیری می نماید.

سرآیند Cache-Control: no-store در پاسخ ها قرار داده شده است تا Client ها یا واسطه های شبکه پاسخ های سنسورها را بدون کنترل ذخیره نکنند. این تصمیم در سامانه های مانیتورینگ مناسب است، زیرا داده های سنسور ماهیت زمانی دارند و استفاده از پاسخ قدیمی می تواند باعث نمایش وضعیت نادرست شود.

API با زبان ++C و استاندارد C++17 نوشته شده و سرور HTTP آن بر پایه Mongoose نسخه ۷.۲۲ است. سه پارامتر مورد نیاز شامل نام سنسور، شناسه سنسور و تاریخ دریافت می شوند. مقادیر از پایگاه داده SQLite خوانده شده و در قالب JSON منظم و قابل خواندن بازگردانده می شوند. حالت نبود داده با آرایه خالی و پیام مناسب مدیریت شده است. کامپایل پروژه از طریق Makefile انجام می شود و اسکریپت Bash مستقل برای کامپایل و اجرای برنامه وجود دارد. اسکریپت ایجاد پایگاه داده موقت نیز برای تست مستقل لایه SQLite در پروژه قرار گرفته است. فایل Markdown کامل شامل وابستگی ها، تنظیمات، ساختار پایگاه داده، روش کامپایل، اجرا و تست تهیه شده است.

IP، Port و مسیر فایل پایگاه داده در کد ثابت نیستند و از فایل پیکربندی یا آرگومان خط فرمان دریافت می شوند. پاسخ ها نیز از داده های واقعی SQLite تولید می شوند و هیچ مقدار ثابتی به عنوان پاسخ API در کد قرار نگرفته است. آزمایش شبکه نشان داد Host می تواند با استفاده از IP فعلی Master به API متصل شود. تست های ورودی معتبر، نبود داده، تاریخ نامعتبر، پارامتر ناقص، عدم تطابق نام و شناسه، متد غیرمجاز و مسیر نامعتبر نیز با موفقیت انجام شدند. نسخه فعلی API برای محیط آموزشی و شبکه محلی طراحی شده است. مهم ترین محدودیت آن نبود HTTPS و احراز هویت است. در نسخه عملی می توان سرویس را پشت Reverse Proxy قرار داد، ارتباط را رمزنگاری کرد و از Token برای کنترل دسترسی استفاده نمود.

در بخش پنجم تمرین، یک API کامل برای خواندن لاگ های روزانه سنسورها طراحی، پیاده سازی و آزمایش شد. سرور با زبان ++C و کتابخانه Mongoose توسعه یافت و اطلاعات مورد نیاز را از پایگاه داده SQLite ماشین Master خواند. ساختار API به گونه ای طراحی شد که اپراتور با ارسال نام سنسور، شناسه و تاریخ، تمام مقادیر ثبت شده در آن روز را به ترتیب زمانی دریافت کند. پاسخ ها در قالب JSON ارائه شدند و حالت های مختلف خطا با کدهای استاندارد HTTP مدیریت گردیدند. فرایند تست از بررسی ساختار پروژه و پایگاه داده آغاز شد و سپس کامپایل، تست مستقل SQLite، اجرای اسکریپت Bash، فعال بودن Port، Health Check، ارتباط Master با Host، بازیابی داده انواع سنسورها، نبود داده، ورودی نامعتبر، مسیر و متد اشتباه، اسکریپت خودکار، ذخیره نتایج و توقف صحیح برنامه بررسی شدند. نتایج تمام آزمایش ها موفقیت آمیز بودند API. مقادیر واقعی موجود در پایگاه داده را بدون استفاده از پاسخ ثابت برگرداند، از طریق شبکه محلی در دسترس بود و در برابر ورودی های نادرست پاسخ های واضح و قابل پیش بینی ارائه کرد. در نتیجه، تمامی الزامات فنی و تحویلی بخش پنجم تمرین با موفقیت پوشش داده شدند.